Reporte

Nombre: Cesar Maximiliano Peña Mendoza Diego Alberto Sierra Mendoza Arturo Sebastian Pérez Zapata	**Matrícula**: 2144135 2143733 2142942

Nombre del curso: Componentes de Lógica de Negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Unidad: 4	**Actividad**: Práctica guiada · Actividad 4 (TechStore)
Fecha: 28/10/2026	

Bibliografía (Formato APA)

Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.

Facultad de Ingeniería Mecánica y Eléctrica, Universidad Autónoma de Nuevo León. (2026). *Práctica guiada · Actividad 4: Implementación del núcleo de negocio TechStore* [Material de clase]. Componentes de Lógica de Negocio, Unidad 4.

Fielding, R., Nottingham, M., & Reschke, J. (2022). *HTTP semantics* (RFC 9110). Internet Engineering Task Force. https://www.rfc-editor.org/info/rfc9110

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Mermaid. (s. f.). *Mermaid documentation*. https://mermaid.js.org/

pytest-dev. (s. f.). *pytest documentation*. https://docs.pytest.org/

Python Software Foundation. (s. f.). *decimal — Decimal fixed point and floating point arithmetic*. Python 3 documentation. https://docs.python.org/3/library/decimal.html

Ramírez, S. (s. f.). *FastAPI*. https://fastapi.tiangolo.com/

Objetivo:

Objetivo general. Implementar y demostrar el núcleo de negocio de la tienda ficticia TechStore: leer clientes, productos y pedidos desde archivos CSV, convertirlos en objetos de dominio, aplicar reglas de descuento con Specifications, controlar el ciclo de vida del pedido con una máquina de estados y Domain Events, y exponer las operaciones mediante una API REST con FastAPI, verificada con pytest.

Objetivos específicos:

1. Modelar Money, Customer, OrderItem y Order con reglas que impidan datos inválidos.
2. Implementar Specifications y componerlas con AND, OR y NOT.
3. Controlar los estados del pedido y registrar los eventos OrderConfirmed, OrderShipped, OrderCancelled, OrderDelivered y OrderReturned.
4. Exponer la API con respuestas HTTP apropiadas (200, 201, 404, 409 y 422).
5. Ejecutar y documentar los escenarios E01 a E15.
6. Automatizar la verificación con pytest y documentar el proyecto en GitHub.

Procedimiento:

1. Entorno de trabajo

Elemento	Detalle
Lenguaje	Python 3.11 o superior (verificado con Python 3.12.3).
API web	FastAPI 0.162.0 con uvicorn.
Pruebas	pytest 9.2.0 y httpx 0.28.1 (TestClient de FastAPI).
Datos de entrada	Cinco archivos CSV del paquete CLN_U4_Paquete_Alumno.zip, sin modificar.
Persistencia	Ninguna: los objetos viven en memoria y los cambios se pierden al cerrar el programa.
Control de versiones	Git y GitHub. El archivo .gitignore excluye .venv, *pycache* y .pytest_cache.

2. Estructura del proyecto

Archivo o carpeta	Responsabilidad
datos/	Cinco CSV del paquete (clientes, productos, pedidos, detalle de pedidos y escenarios).
cargar_csv.py	Cargador provisto por el paquete: convierte los CSV en objetos Order. No se modificó.
app/domain.py	Money, Customer, OrderItem, Order, OrderStatus, DomainEvent y las excepciones de dominio.
app/specifications.py	Patrón Specification (And, Or, Not), reglas concretas y cálculo del descuento.
app/api.py	Controladores FastAPI: buscan el pedido, llaman al método de Order y responden. Sin reglas de negocio.
tests/test_domain.py	36 pruebas de Money, OrderItem, Order, Specifications y máquina de estados.
tests/test_escenarios.py	22 pruebas: una por cada escenario E01 a E15 y casos de descuento parametrizados.
tests/test_api.py	19 pruebas de la API con TestClient (códigos 200, 201, 404, 409 y 422).
ejecutar_escenarios.py	Ejecuta E01 a E15 con datos recién cargados y genera la bitácora docs/BITACORA.md.
docs/	BITACORA.md, Reporte_TechStore_U4_Equipo7.pdf y docs/diagramas/ con el diagrama de la máquina de estados en formato .mmd.
README.md, requirements.txt	Instalación, ejecución, reglas y endpoints; dependencias del proyecto.

3. Datos de entrada

Archivo	Registros	Campos	Uso
clientes.csv	12	cliente_id, nombre, es_vip, activo	es_vip y activo: 1 = sí, 0 = no. Los clientes inactivos no pueden confirmar pedidos.

Reporte

Archivo	Registros	Campos	Uso
productos.csv	12	sku, nombre, precio_mxn, existencia	Precios en Decimal. La existencia es solo información de catálogo (no hay control de inventario).
pedidos.csv	16	pedido_id, cliente_id, tiene_promocion, estado_inicial, descripcion	Todos inician en DRAFT.
detalle_pedidos.csv	21	pedido_id, sku, cantidad	Renglones de cada pedido.
escenarios.csv	15	caso_id, pedido_id, accion, objetivo, pista	Casos E01 a E15 a ejecutar.

Relación	Campo de enlace
pedidos.csv → clientes.csv	cliente_id
detalle_pedidos.csv → pedidos.csv	pedido_id
detalle_pedidos.csv → productos.csv	sku (el precio unitario sale de productos.csv)

Pedidos cargados con python cargar_csv.py (todos en DRAFT):

Pedido	Cliente	VIP	Activo	Promo	Uds	Renglones	Total MXN	Estado
O001	C001 (Ana Torres)	Sí	Sí	No	4	2	2100.00	DRAFT
O002	C002 (Luis Medina)	No	Sí	No	2	2	1050.00	DRAFT
O003	C003 (Carla Ruiz)	Sí	No	No	4	2	2100.00	DRAFT
O004	C004 (Diego Peña)	No	No	No	1	1	850.00	DRAFT
O005	C005 (Sofia Lara)	Sí	Sí	Sí	4	2	2100.00	DRAFT
O006	C006 (Pedro Soto)	No	Sí	No	1	1	18000.00	DRAFT
O007	C007 (Mariana Gil)	Sí	Sí	No	0	0	0.00	DRAFT
O008	C008 (Raul Vega)	No	Sí	No	20	1	2400.00	DRAFT
O009	C009 (Elena Mora)	Sí	Sí	No	19	1	2280.00	DRAFT
O010	C010 (Jorge Luna)	No	Sí	No	2	2	500.00	DRAFT
O011	C011 (Valeria Rios)	Sí	Sí	No	3	2	1500.00	DRAFT
O012	C012 (Oscar Nieto)	No	Sí	No	1	1	850.00	DRAFT
O013	C001 (Ana Torres)	Sí	Sí	No	1	1	600.00	DRAFT
O014	C002 (Luis Medina)	No	Sí	No	1	1	600.00	DRAFT
O015	C005 (Sofia Lara)	Sí	Sí	No	1	1	450.00	DRAFT
O016	C007 (Mariana Gil)	Sí	Sí	No	1	1	2100.00	DRAFT

Carga de datos



Actividad corta: cálculo manual frente al resultado del programa.

Pedido	Cliente	Unidades	Total manual	Total del programa
O001	C001 (VIP)	4 (2 × SKU001 + 2 × SKU002)	2 × 600 + 2 × 450 = 2100.00	2100.00
O006	C006	1 (1 × SKU008)	1 × 18000 = 18000.00	18000.00
O010	C010	2 (SKU003 + SKU009)	120 + 380 = 500.00	500.00

4. Diseño del dominio (app/domain.py)

Elemento	Tipo (Evans, 2003)	Reglas que impone
Money	Value Object inmutable	Guarda amount como Decimal y currency (MXN por defecto). Rechaza importes negativos, NaN, infinitos y valores que no sean Decimal (por ejemplo float). add() rechaza monedas distintas y devuelve un Money nuevo. rounded() redondea a centavos.
Customer	Entidad inmutable	id, es_vip y activo. Un cliente inactivo no puede confirmar pedidos.
OrderItem	Value Object inmutable	SKU no vacío y cantidad entera mayor que 0. El subtotal se calcula (precio unitario × cantidad), no se captura.
Order	Aggregate Root	Concentra las reglas: items, status, events, tracking_number y cancel_reason. units suma cantidades y total suma subtotales (redondeado a centavos). add_item() solo en DRAFT, máximo 20 unidades y solo MXN.
OrderStatus	Enumeración	DRAFT, CONFIRMED, SHIPPED, DELIVERED, CANCELLED y RETURNED.
DomainEvent	Evento inmutable	name, order_id, occurred_at (UTC) y detail (guía o motivo).

Excepción de dominio	Cuándo se lanza	HTTP en la API
DomainException	Clase base de las excepciones del dominio.	—
InvalidOrderData	Monto, artículo, guía o motivo inválidos; pedido vacío o cliente inactivo al confirmar.	422
InvalidOrderTransition	Operación no permitida desde el estado actual.	409
OrderItemLimitExceeded	La suma de unidades superaría 20.	422

Regla de negocio	Dónde se implementa	Pruebas
Money sin negativos, con Decimal	Money.*post_init*	test_money_negativo, test_money_rechaza_float_y_no_finitos

Regla de negocio	Dónde se implementa	Pruebas
Máximo 20 unidades (suma, no renglones)	Order.add_item	test_20_unidades_exactas_permitidas, test_21_unidades_rechazadas_sin_modificar_items, test_limite_cuenta_unidades_no_renglones
Solo DRAFT admite artículos	Order.add_item	test_agregar_item_solo_en_draft
Todos los pedidos inician en DRAFT	Carga desde CSV	test_carga_16_pedidos_todos_draft
Descuento 10 %: VIP, total > 500, ≥ 4 unidades, sin promoción	ten_percent	test_e01_descuento_vip_10, test_frontera_monto_estricto_con_vip_y_4_unidades, test_frontera_unidades_con_vip
Descuento 5 %: total > 5000, sin promoción	five_percent	test_e05_descuento_alternativo_5, test_frontera_5000_estricta
Prioridad del 10 % sobre el 5 %	discount_rate	test_prioridad_10_sobre_5
Sin control de inventario	Por diseño: existencia es solo contexto de catálogo	No aplica
Sin persistencia entre ejecuciones	Diccionario en memoria en app/api.py	Fixture fixtures_limpios en test_api.py

5. Specification Pattern (app/specifications.py)

Specification	Condición que evalúa	Parámetro
Specification	Clase base: is_satisfied_by(order) y operadores & (AND), | (OR) y ~ (NOT). Solo responde verdadero o falso; no modifica el pedido.	—
AndSpec / OrSpec / NotSpec	Combinan dos Specifications (o niegan una).	left, right / inner
MinimumAmountSpec	Total estrictamente mayor que el umbral (>, no ≥).	500 y 5000
VIPCustomerSpec	El cliente del pedido es VIP.	—
MinimumUnitsSpec	El pedido tiene al menos N unidades (≥).	4
ActivePromotionSpec	El pedido tiene promoción activa.	—
ActiveCustomerSpec	Quinta Specification: el cliente está activo. Disponible para validaciones; no participa en las reglas de descuento.	—

Regla	Expresión
ten_percent	`MinimumAmountSpec(Decimal("500")) & VIPCustomerSpec() & MinimumUnitsSpec(4) & ~ActivePromotionSpec()`
five_percent	`MinimumAmountSpec(Decimal("5000")) & ~ActivePromotionSpec()`
any_discount	`ten_percent | five_percent`
discount_rate(order)	`0.10 si cumple ten_percent; si no, 0.05 si cumple five_percent; si no, 0.`

Evaluación de descuento de los 16 pedidos (resultado real del programa):

Pedido	Total	Tasa	Descuento	Total neto	Motivo
O001	2100.00	10%	210.00	1890.00	VIP, total > 500, 4 unidades y sin promoción: cumple el 10 %.
O002	1050.00	0%	0.00	1050.00	No es VIP y 1050.00 no supera 5000.
O003	2100.00	10%	210.00	1890.00	Cumple el 10 %. El cliente inactivo no cambia la tasa; bloquea la confirmación (E03).
O004	850.00	0%	0.00	850.00	No es VIP y 850.00 no supera 5000.
O005	2100.00	0%	0.00	2100.00	Promoción activa: bloquea ambos descuentos.
O006	18000.00	5%	900.00	17100.00	Total 18000.00 > 5000 y sin promoción: cumple el 5 %.
O007	0.00	0%	0.00	0.00	Sin artículos: total 0.00.
O008	2400.00	0%	0.00	2400.00	No es VIP y 2400.00 no supera 5000.
O009	2280.00	10%	228.00	2052.00	VIP, 2280.00 > 500, 19 unidades y sin promoción: cumple el 10 %.
O010	500.00	0%	0.00	500.00	Total exactamente 500.00 (no supera 500) y cliente no VIP.
O011	1500.00	0%	0.00	1500.00	VIP con total > 500, pero solo 3 unidades (< 4).
O012	850.00	0%	0.00	850.00	No es VIP y 850.00 no supera 5000.
O013	600.00	0%	0.00	600.00	VIP con 600.00 > 500, pero solo 1 unidad.
O014	600.00	0%	0.00	600.00	No es VIP y 600.00 no supera 5000.
O015	450.00	0%	0.00	450.00	VIP, pero 450.00 no supera 500 y 1 unidad.
O016	2100.00	0%	0.00	2100.00	VIP con 2100.00 > 500, pero solo 1 unidad.

6. Máquina de estados y Domain Events

Regla común de cada método: validar primero, cambiar el estado después y registrar el evento al final.

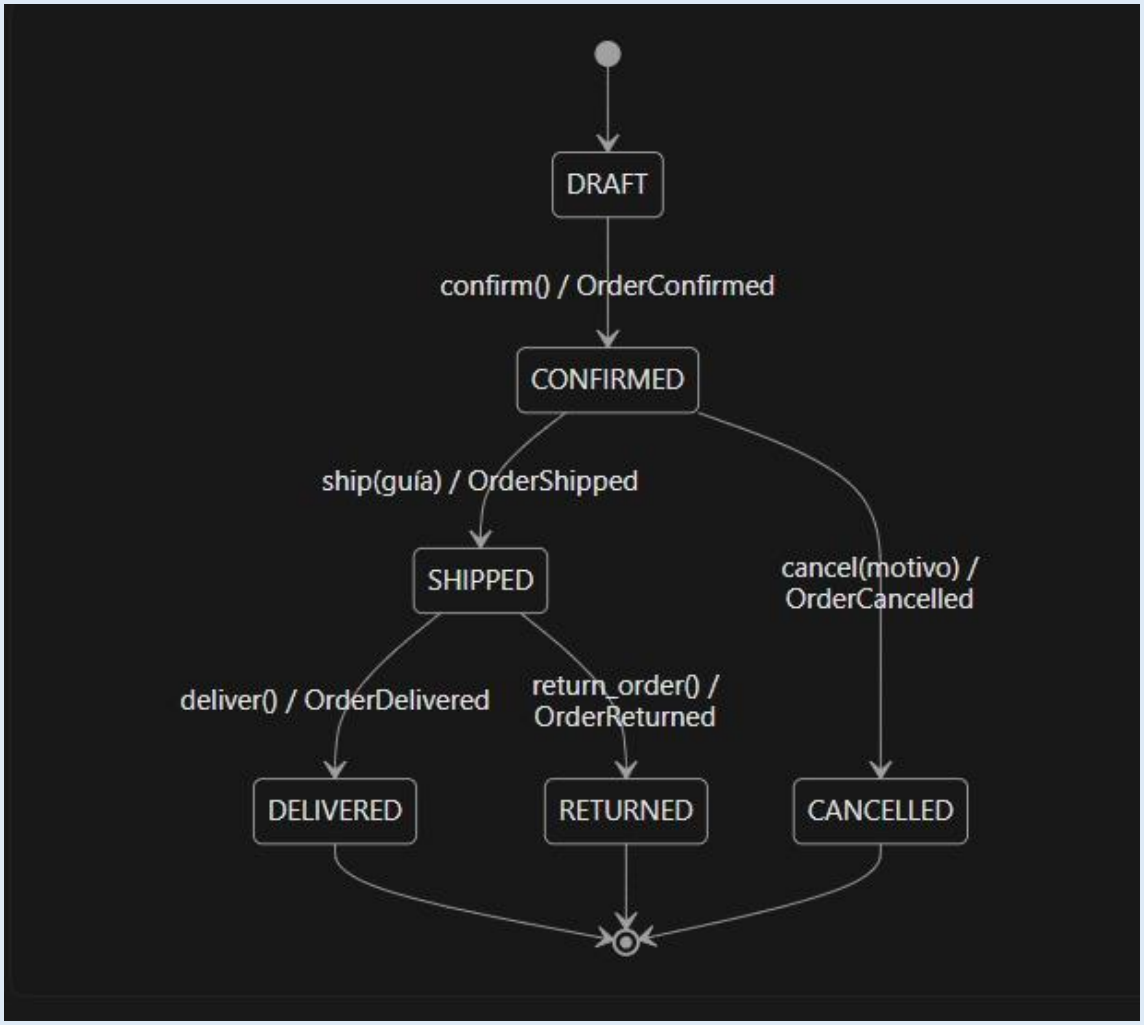
Método	Estado requerido	Validación adicional	Evento	Estado nuevo	Si falla
confirm()	DRAFT	Al menos un artículo y cliente activo	OrderConfirmed	CONFIRMED	409 por estado; 422 por datos
ship(guía)	CONFIRMED	Guía no vacía	OrderShipped	SHIPPED	409 por estado; 422 sin guía
cancel(motivo)	CONFIRMED	Motivo no vacío	OrderCancelled	CANCELLED	409 por estado; 422 sin motivo
deliver()	SHIPPED	—	OrderDelivered	DELIVERED	409
return_order()	SHIPPED	—	OrderReturned	RETURNED	409

Evento	Cuándo se emite	Datos que registra
OrderConfirmed	Después de pasar de DRAFT a CONFIRMED.	name, order_id, occurred_at (UTC).
OrderShipped	Después de pasar de CONFIRMED a SHIPPED.	Además, detail con el número de guía.

Evento	Cuándo se emite	Datos que registra
OrderCancelled	Después de pasar de CONFIRMED a CANCELLED.	Además, detail con el motivo.
OrderDelivered	Después de pasar de SHIPPED a DELIVERED.	name, order_id, occurred_at (UTC).
OrderReturned	Después de pasar de SHIPPED a RETURNED.	name, order_id, occurred_at (UTC).

Diagrama 1. Máquina de estados del pedido

Propósito: *muestra los seis estados, las transiciones permitidas y el evento emitido en cada una.*



Máquina de estados del pedido

Qué es un Domain Event. Un hecho ya ocurrido en el dominio. Queda en order.events como historial auditable del pedido.

Por qué solo tras una transición válida. Si se registrara antes de validar, quedaría el rastro de un hecho que nunca sucedió (por ejemplo un OrderShipped sin guía). Una operación rechazada no cambia el estado ni agrega eventos (E03, E06, E11, E12 y E13).

Estados terminales. DELIVERED, CANCELLED y RETURNED no admiten más operaciones: cualquier intento lanza InvalidOrderTransition.

Uso posterior. La lista de eventos es el punto desde el cual otros componentes podrían reaccionar (notificaciones, facturación, inventario) sin acoplarse al pedido.

7. API REST (app/api.py)

Método	Ruta	Cuerpo	Éxito	Errores posibles
GET	/orders	–	200	—
GET	/orders/{id}	–	200	404
GET	/orders/{id}/discount	–	200	404
POST	/orders/{id}/items	`{"sku": "SKU003", "quantity": 1}`	201	404 (pedido o SKU), 409, 422
POST	/orders/{id}/confirm	–	200	404, 409, 422
POST	/orders/{id}/ship	`{"tracking_number": "TRK-1"}`	200	404, 409, 422
POST	/orders/{id}/cancel	`{"reason": "motivo"}`	200	404, 409, 422
POST	/orders/{id}/deliver	–	200	404, 409
POST	/orders/{id}/return	–	200	404, 409

Código HTTP	Causa	Motivo del mapeo (RFC 9110)
404	Pedido o SKU inexistente	El recurso solicitado no existe.
409	InvalidOrderTransition	La operación entra en conflicto con el estado actual del pedido.
422	InvalidOrderData y OrderItemLimitExceeded	La petición es correcta, pero incumple una regla de negocio. Decisión de diseño: el límite de 20 unidades se trata como 422 porque la práctica no define su código.

Cómo acceder a la API y a la página de pruebas (Swagger):

Paso 1. Abrir una terminal dentro de la carpeta del proyecto (la que contiene app/ y cargar_csv.py).

```
cd techstore-u4
```

Resultado esperado: El prompt queda en la raíz del proyecto.

Paso 2. Crear y activar el entorno virtual.

```
python -m venv .venv
Windows PowerShell: .venv\Scripts\Activate.ps1
macOS o Linux: source .venv/bin/activate
```

Resultado esperado: El prompt muestra (.venv).

Paso 3. Instalar dependencias.

```
python -m pip install -r requirements.txt
```

Resultado esperado: Se instalan fastapi, uvicorn, pytest y httpx.

Paso 4. Iniciar el servidor.

```
uvicorn app.api:app --reload
```

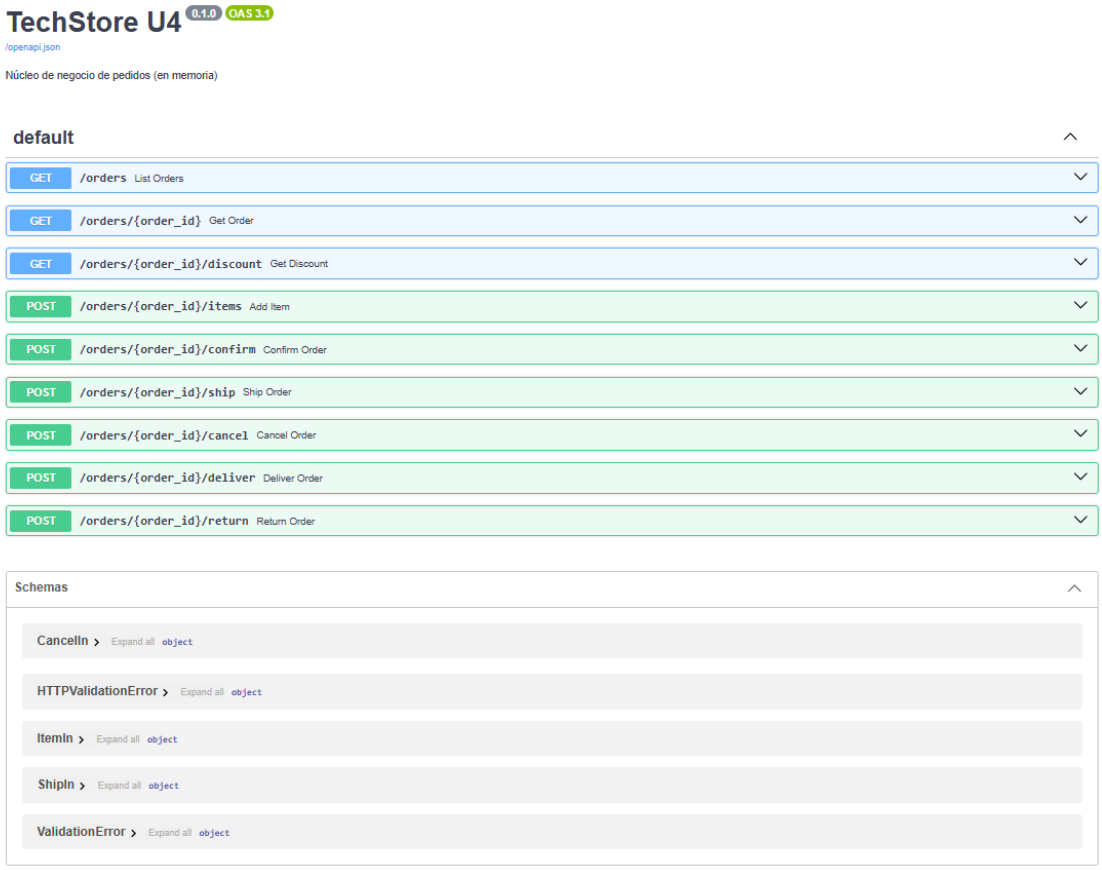
Resultado esperado: La terminal indica Uvicorn running on http://127.0.0.1:8000.

Paso 5. Abrir Swagger en el navegador.

```
http://127.0.0.1:8000/docs
```

Resultado esperado: Aparece la lista de endpoints de TechStore U4.

Vista General



Paso 6. Ver el contrato OpenAPI (opcional).

```
http://127.0.0.1:8000/openapi.json
```

Resultado esperado: Se muestra el JSON del contrato.

Paso 7. Probar un endpoint: abrirlo, pulsar Try it out, escribir el id (por ejemplo O013) y pulsar Execute.

```
POST /orders/O001/confirm
```

Resultado esperado: Code 200 con {"id": "O013", "status": "CONFIRMED"}.



Paso 8. Repetir la misma llamada para provocar el rechazo.

```
POST /orders/O001/confirm
```

Resultado esperado: Code 409 con {"detail": "Solo DRAFT puede confirmarse"}.



Paso 9. Reiniciar los datos entre pruebas: detener el servidor con Ctrl+C y volver a ejecutar el paso 4.

```
uvicorn app.api:app --reload
```

Resultado esperado: Los 16 pedidos vuelven a DRAFT (los datos viven solo en memoria).

Casos adicionales de Swagger para las capturas

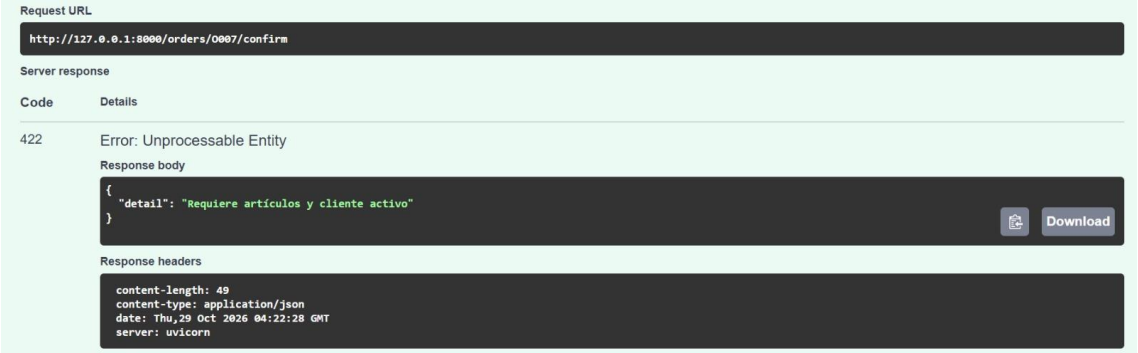
Los casos del pedido O013 (200 y luego 409) ya corresponden a las capturas 5 y 6. Los siguientes se prueban con otros pedidos:

Pedido NOEXISTE. confirmar un id inexistente devuelve 404.



Reporte

Pedido O007. confirmar un pedido sin artículos devuelve 422.



Pedido O012. confirmar (200) y después enviar con tracking_number vacío devuelve 422.



8. Bitácora de escenarios E01 a E15

Cada escenario parte de pedidos recién cargados con load_fixture(); no se editaron pedidos ni reglas. Resultado: 15 de 15 escenarios coinciden con el objetivo de escenarios.csv.

Caso	Precondiciones (estado inicial)	Operación	Resultado real	Evento emitido	Explicación	Prueba (tests/test_escenarios.py)
E01 O001	DRAFT; cliente C001 (VIP, activo); 4 uds; total 2100.00 MXN; promoción no	discount_rate(order) / discount_amount(order)	tasa 10%, descuento 210.00, neto 1890.00; estado DRAFT	Ninguno (consulta: no cambia el estado)	Cumple las 4 condiciones del 10 %: VIP, 2100.00 > 500, 4 ≥ 4 unidades y sin promoción.	test_e01_descuento_vip_10
E02 O002	DRAFT; cliente C002 (no VIP, activo); 2 uds; total 1050.00 MXN; promoción no	discount_rate(order) / discount_amount(order)	tasa 0%, descuento 0.00, neto 1050.00; estado DRAFT	Ninguno (consulta: no cambia el estado)	C002 no es VIP (falla la regla de 10 %) y 1050.00 no supera 5000 (falla la de 5 %).	test_e02_no_vip_sin_descuento
E03 O003	DRAFT; cliente C003 (VIP, inactivo); 4 uds; total 2100.00 MXN; promoción no	confirm()	confirm() → InvalidOrderData: Requiere artículos y cliente activo | estado final DRAFT	Ninguno (operación rechazada)	C003 es VIP pero está inactivo: confirm() valida antes de cambiar el estado y no emite evento.	test_e03_cliente_inactivo_rechaza_confirmacion
E04 O005	DRAFT; cliente C005 (VIP, activo); 4 uds; total 2100.00 MXN; promoción sí	discount_rate(order) / discount_amount(order)	tasa 0%, descuento 0.00, neto 2100.00; estado DRAFT	Ninguno (consulta: no cambia el estado)	C005 es VIP, 4 unidades y 2100.00, pero la promoción activa bloquea ambos descuentos (~ActivePromotionSpec).	test_e04_promocion_activa_bloquea_descuento
E05 O006	DRAFT; cliente C006 (no VIP, activo); 1 uds; total 18000.00 MXN; promoción no	discount_rate(order) / discount_amount(order)	tasa 5%, descuento 900.00, neto 17100.00; estado DRAFT	Ninguno (consulta: no cambia el estado)	18000.00 > 5000 y sin promoción cumple el 5 %; C006 no es VIP, por eso no alcanza el 10 %.	test_e05_descuento_alternativo_5
E06 O007	DRAFT; cliente C007 (VIP, activo); 0 uds; total 0.00 MXN; promoción no	confirm()	confirm() → InvalidOrderData: Requiere artículos y cliente activo | estado final DRAFT	Ninguno (operación rechazada)	Sin renglones, confirm() lanza InvalidOrderData; el pedido sigue DRAFT y no hay evento.	test_e06_pedido_vacio_rechaza_confirmacion
E07 O008	DRAFT; cliente C008 (no VIP, activo); 20 uds; total 2400.00 MXN; promoción no	confirm()	confirm() → OK | estado final CONFIRMED	OrderConfirmed	20 unidades es el máximo permitido (solo > 20 se rechaza); el pedido se confirma.	test_e07_20_unidades_permitidas_y_confirma
E08 O009	DRAFT; cliente C009 (VIP, activo); 19 uds; total 2280.00 MXN; promoción no	add_item(OrderItem('SKU003', 2, 120.00))	add_item(SKU003 × 2) → OrderItemLimitExceeded: Máximo 20 unidades; uds=19	Ninguno (operación rechazada)	19 + 2 = 21 > 20: add_item() detiene la operación antes de modificar items. Se suman unidades, no renglones.	test_e08_21_unidades_exceden_limite
E09 O010	DRAFT; cliente C010 (no VIP, activo); 2 uds; total 500.00 MXN; promoción no	discount_rate(order) / discount_amount(order)	tasa 0%, descuento 0.00, neto 500.00; estado DRAFT	Ninguno (consulta: no cambia el estado)	El total 500.00 no supera 500 (umbral estricto >). Nota: C010 no es VIP y tiene 2 unidades, así que este pedido solo no aísla la frontera; la frontera se prueba con VIP y 4 unidades en test_frontera_monto_estricto_con_vip_y_4_unidades.	test_e09_umbral_estricto_500

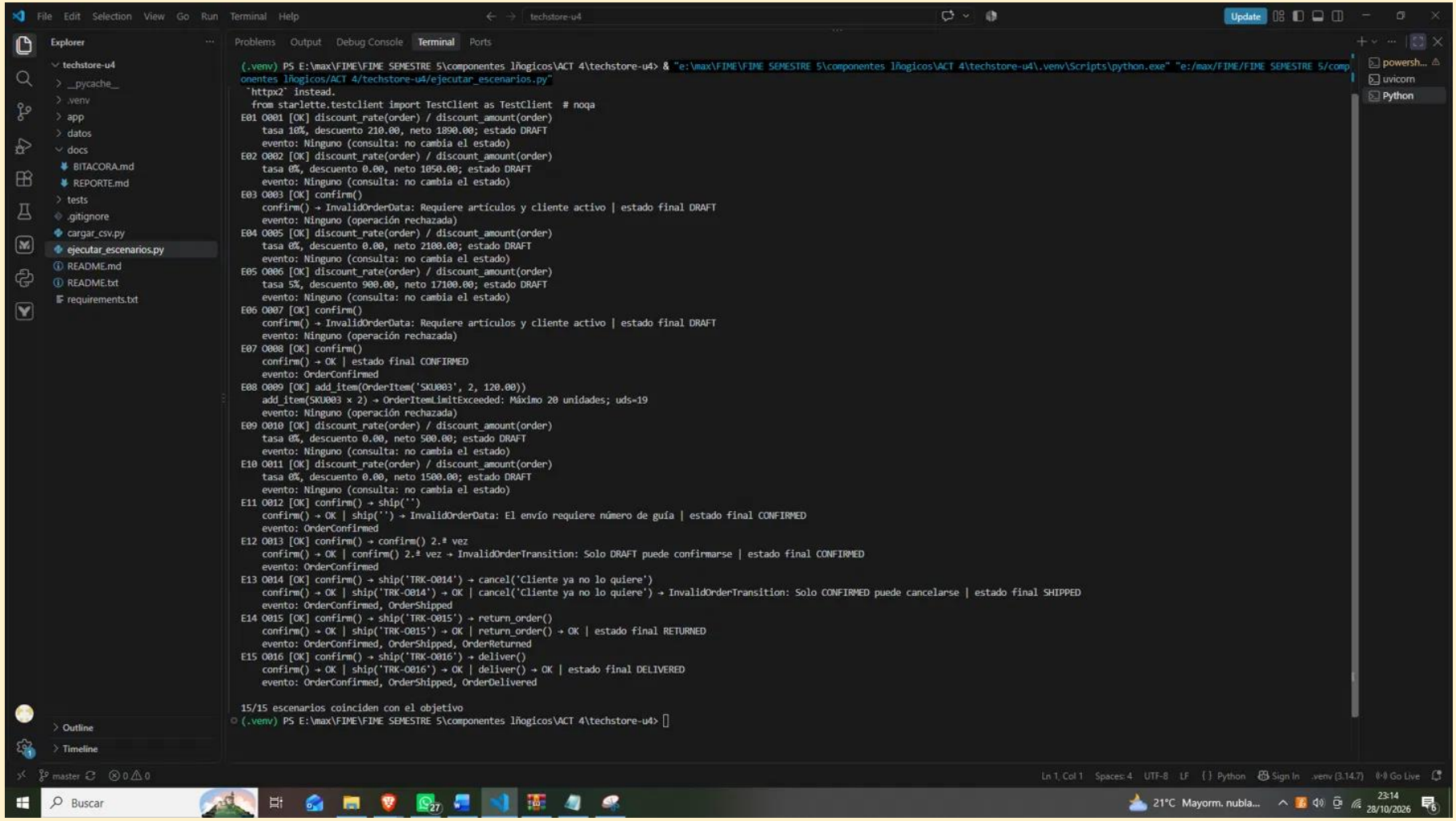
Caso	Precondiciones (estado inicial)	Operación	Resultado real	Evento emitido	Explicación	Prueba (tests/test_escenarios.py)
E10 O011	DRAFT; cliente C011 (VIP, activo); 3 uds; total 1500.00 MXN; promoción no	discount_rate(order) / discount_amount(order)	tasa 0%, descuento 0.00, neto 1500.00; estado DRAFT	Ninguno (consulta: no cambia el estado)	C011 es VIP y 1500.00 > 500, pero 3 unidades < 4: falla MinimumUnitsSpec(4).	test_e10_vip_con_3_unidades_sin_descuento
E11 O012	DRAFT; cliente C012 (no VIP, activo); 1 uds; total 850.00 MXN; promoción no	confirm() → ship('')	confirm() → OK | ship('') → InvalidOrderData: El envío requiere número de guía | estado final CONFIRMED	OrderConfirmed	La confirmación es válida; ship('') lanza InvalidOrderData. Sigue CONFIRMED y no se emite OrderShipped.	test_e11_envio_sin_guia_rechazado
E12 O013	DRAFT; cliente C001 (VIP, activo); 1 uds; total 600.00 MXN; promoción no	confirm() → confirm() 2.ª vez	confirm() → OK | confirm() 2.ª vez → InvalidOrderTransition: Solo DRAFT puede confirmarse | estado final CONFIRMED	OrderConfirmed	La segunda confirmación lanza InvalidOrderTransition (solo DRAFT confirma) y no agrega un segundo evento.	test_e12_segunda_confirmacion_rechazada
E13 O014	DRAFT; cliente C002 (no VIP, activo); 1 uds; total 600.00 MXN; promoción no	confirm() → ship('TRK-O014') → cancel('Cliente ya no lo quiere')	confirm() → OK | ship('TRK-O014') → OK | cancel('Cliente ya no lo quiere') → InvalidOrderTransition: Solo CONFIRMED puede cancelarse | estado final SHIPPED	OrderConfirmed, OrderShipped	cancel() exige CONFIRMED; desde SHIPPED lanza InvalidOrderTransition y no emite OrderCancelled.	test_e13_cancelacion_desde_shipped_rechazada
E14 O015	DRAFT; cliente C005 (VIP, activo); 1 uds; total 450.00 MXN; promoción no	confirm() → ship('TRK-O015') → return_order()	confirm() → OK | ship('TRK-O015') → OK | return_order() → OK | estado final RETURNED	OrderConfirmed, OrderShipped, OrderReturned	SHIPPED permite return_order(): estado final RETURNED y tres eventos en orden.	test_e14_devolucion_termina_returned
E15 O016	DRAFT; cliente C007 (VIP, activo); 1 uds; total 2100.00 MXN; promoción no	confirm() → ship('TRK-O016') → deliver()	confirm() → OK | ship('TRK-O016') → OK | deliver() → OK | estado final DELIVERED	OrderConfirmed, OrderShipped, OrderDelivered	SHIPPED permite deliver(): estado final DELIVERED y tres eventos en orden.	test_e15_entrega_termina_delivered

Operación equivalente en la API (código HTTP obtenido al ejecutarla con TestClient sobre datos recién cargados):

Caso	Petición → código
E01	GET /orders/O001/discount → 200 tasa 0.10
E02	GET /orders/O002/discount → 200 tasa 0
E03	POST /orders/O003/confirm → 422
E04	GET /orders/O005/discount → 200 tasa 0
E05	GET /orders/O006/discount → 200 tasa 0.05
E06	POST /orders/O007/confirm → 422
E07	POST /orders/O008/confirm → 200

Caso	Petición → código
E08	POST /orders/O009/items → 422
E09	GET /orders/O010/discount → 200 tasa 0
E10	GET /orders/O011/discount → 200 tasa 0
E11	POST /orders/O012/confirm → 200
	POST /orders/O012/ship → 422
E12	POST /orders/O013/confirm → 200
	POST /orders/O013/confirm → 409
E13	POST /orders/O014/confirm → 200
	POST /orders/O014/ship → 200
	POST /orders/O014/cancel → 409
E14	POST /orders/O015/confirm → 200
	POST /orders/O015/ship → 200
	POST /orders/O015/return → 200
E15	POST /orders/O016/confirm → 200
	POST /orders/O016/ship → 200
	POST /orders/O016/deliver → 200

Escenarios E01 a E15



9. Pruebas automáticas con pytest

Archivo	Pruebas	Qué demuestra
tests/test_domain.py	36	Money (negativos, float, NaN, inmutabilidad), OrderItem, carga de los 16 pedidos, límite de 20 unidades, confirmación, envío, cancelación, entrega, devolución, estados terminales y Specifications (fronteras y prioridad).
tests/test_escenarios.py	22	Una prueba por cada escenario E01 a E15, más casos de descuento parametrizados.
tests/test_api.py	19	Códigos 200, 201, 404, 409 y 422; que una operación rechazada no agrega eventos; endpoint de descuento; /docs y /openapi.json.
Total	**77**	Resultado de python -m pytest -q: 77 passed, sin fallos.

Verificación de la calidad de las pruebas: se introdujo un error a propósito y se comprobó que la suite lo detecta.

Error introducido	Pruebas que fallaron	Qué protege
Cambiar > por ≥ en MinimumAmountSpec	2 de 77	El umbral estricto de 500 y de 5000. Nota: E09 por sí solo no falla con este error, porque O010 no es VIP y tiene 2 unidades; por eso se agregaron pruebas de frontera con VIP y 4 unidades.
Limitar por número de renglones en lugar de unidades	4 de 77	El límite de 20 se calcula sobre la suma de unidades.
Registrar OrderShipped antes de validar la guía	8 de 77	El orden validar, cambiar estado y registrar evento.
Código original restaurado	0 de 77	Suite completa en verde.

PRUEBAS AUTOMÁTICAS

```
(.venv) PS C:\Users\arsep\OneDrive\Desktop\UNIDAD 4 ComLogNeg\CLN_U4_Paquete_Alumno\Paquete_TechStore_Alumno> $env:PYTHONPATH="."; py
test
============================================================ test session starts =============================================================
platform win32 -- Python 3.12.10, pytest-9.2.0, pluggy-1.7.0
rootdir: C:\Users\arsep\OneDrive\Desktop\UNIDAD 4 ComLogNeg\CLN_U4_Paquete_Alumno\Paquete_TechStore_Alumno
plugins: anyio-4.15.1
collected 77 items

tests\test_api.py ...................                                                                                                  [ 24%]
tests\test_domain.py ....................................                                                                              [ 71%]
tests\test_escenarios.py ......................                                                                                        [100%]

============================================================== warnings summary ==============================================================
.venv\Lib\site-packages\fastapi\testclient.py:1
  C:\Users\arsep\OneDrive\Desktop\UNIDAD 4 ComLogNeg\CLN_U4_Paquete_Alumno\Paquete_TechStore_Alumno\.venv\Lib\site-packages\fastapi\t
estclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

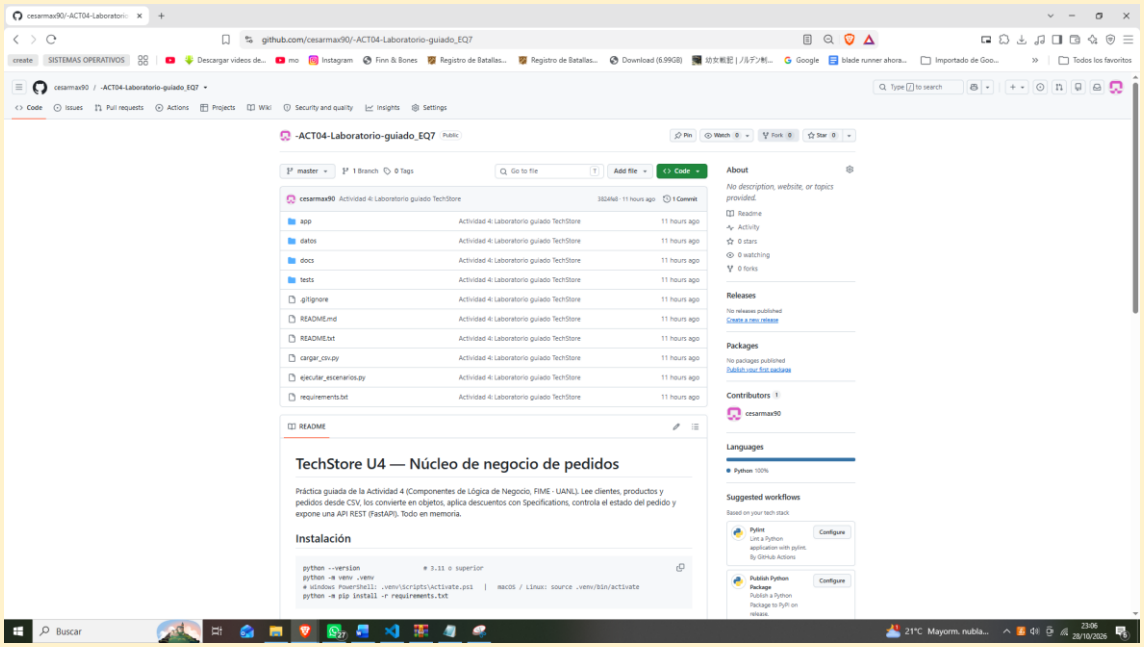
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
```

10. Evidencias (capturas requeridas)

Cada captura se inserta en el recuadro amarillo (o azul, para el diagrama) que aparece en la sección correspondiente. Resumen de dónde va cada una:

Captura 1 · Carga de datos → sección 3, después de la tabla de pedidos cargados.
Captura 2 · Escenarios E01 a E15 → sección 8, después de la tabla de la API.
Captura 3 · Pruebas (77 passed) → sección 9, al final.
Captura 4 · Swagger: vista general → sección 7, dentro del paso 5.
Captura 5 · Swagger: éxito 200 → sección 7, dentro del paso 7.
Captura 6 · Swagger: rechazo 409 → sección 7, dentro del paso 8.
Captura 7 · Swagger: rechazo 404 → sección 7, casos adicionales (NOEXISTE).
Captura 8 · Swagger: rechazo 422 → sección 7, casos adicionales (O007).
Captura 9 · Swagger: envío sin guía → sección 7, casos adicionales (O012).
Captura 10 · Diagrama 1 renderizado → sección 6, después de la tabla de eventos.
Captura 11 · Repositorio en GitHub → aquí mismo, abajo.

Repositorio en GitHub



11. Respuestas a las preguntas del reporte

¿Por qué Decimal en lugar de float para precios?

float es binario y no representa exactamente valores como 0.10 (0.1 + 0.2 ≠ 0.3); esos errores se acumulan en subtotales y descuentos. Decimal trabaja en base 10 y los centavos son exactos (Python Software Foundation, s. f.).

¿Diferencia entre contar renglones y sumar unidades?

O009 tiene un solo renglón con 19 unidades; un pedido de 10 renglones de 2 unidades suma 20. El límite aplica a la suma de cantidades: contar renglones permitiría pedidos enormes con pocos renglones.

¿Qué ventaja tiene encapsular una regla en una Specification?

La regla tiene nombre, se prueba por separado, se reutiliza y se combina con &, | y ~ sin tocar el pedido. Cambiar un umbral implica editar un solo lugar.

¿Por qué los eventos se registran solo después de una transición válida?

Un evento afirma que algo ya ocurrió. Registrarlo antes de validar dejaría hechos falsos en el historial (E11, E12 y E13).

¿Qué código se reutiliza si se reemplaza FastAPI?

Todo app/domain.py, app/specifications.py y cargar_csv.py, además de las pruebas de dominio. Solo app/api.py (rutas, esquemas y traducción a HTTP) se sustituye.

12. Lista de verificación

Criterio	Cómo se comprobó	Estado
Ejecutar python cargar_csv.py sin errores	Ejecución del cargador (16 pedidos)	Cumple
O001 muestra 4 unidades y 2100.00 MXN	test_pedido_o001_inicia_draft	Cumple
Regla del 10 %: VIP, > 500, ≥ 4 unidades y sin promoción	E01, E04, E10 y pruebas de frontera	Cumple
Regla del 5 %: > 5000 y sin promoción	E05 y test_frontera_5000_estricta	Cumple
Una operación inválida no cambia el estado ni agrega eventos	E03, E06, E11, E12 y E13	Cumple
La API responde 404, 409 y 422	tests/test_api.py y llamadas reales con curl	Cumple
pytest termina sin fallos	77 passed	Cumple
El repositorio contiene README y evidencias	README.md y docs/ incluidos; faltan las capturas	Cumple

13. Reflexión: tres errores encontrados

Error 1

Error encontrado. E09 no probaba la frontera de 500 por sí solo: al cambiar > por ≥ seguía pasando, porque O010 no es VIP y tiene 2 unidades.

Cómo se solucionó. Se agregaron pruebas con VIP, 4 unidades y totales de 500.00 y 500.04 (y de 5000.00 y 5000.01).

Qué se aprendió. Un escenario puede dar el resultado esperado por otra razón; conviene aislar cada condición.

Error 2

Error encontrado. Money(600.0) fallaba con AttributeError, un error ajeno al dominio, porque float no tiene is_finite.

Cómo se solucionó. Se agregó una validación de tipo que lanza InvalidOrderData y una prueba.

Qué se aprendió. También se debe validar el tipo del dato, no solo su valor.

Error 3

Error encontrado. Un evento registrado antes de validar pasaba desapercibido sin pruebas específicas.

Cómo se solucionó. Se movió a propósito y fallaron 8 pruebas; esas pruebas de “sin evento tras un rechazo” protegen el orden validar, cambiar y registrar.
Qué se aprendió. Probar también lo que NO debe ocurrir (sin evento, sin cambio de estado).

Conclusiones

Modelo de dominio. Concentrar las reglas en Money, OrderItem y Order impide que existan datos inválidos: la validación ocurre al crear el objeto o al ejecutar la operación, no en la API.

Specifications. Expresar los descuentos como reglas componibles (&, | y ~) hizo que cada condición fuera legible, probable por separado y fácil de ajustar. La prioridad del 10 % sobre el 5 % queda en un solo lugar.

Estados y eventos. Validar antes de cambiar el estado y registrar el evento al final garantiza un historial confiable: las operaciones rechazadas no dejan rastro (E03, E06, E11, E12 y E13).

API REST. Los controladores quedaron delgados: buscan el pedido, llaman al método de Order y traducen cada excepción a 404, 409 o 422. Así la lógica de negocio podría reutilizarse con otra interfaz.

Pruebas. 77 pruebas pasan y detectan los errores introducidos a propósito. Las pruebas de frontera fueron necesarias porque algunos escenarios del paquete no aíslan una sola condición.

Resultado. Los 15 escenarios E01 a E15 coinciden con su objetivo sin cambiar las reglas de la Actividad 4.

Repositorio Github

https://github.com/cesarmax90/-ACT04-Laboratorio-guiado_EQ7.git